

Para: Empresas que precisam rastrear suas encomendas
Que necessita de: Auditoria da entrega realizada ou não, e qualidade da entrega para satisfação do cliente final.

O UNIRASTRO é um: Saas

Que oferece: Rastreamento, dados e relatórios sobre as encomendas enviadas aos clientes.

Diferente de: Site próprio das transportadoras ou soluções online como melhorrastreio.com.br

Nosso produto entregará: O sistema oferecerá à empresa uma área pró ativa, para que antes do cliente reclamar do produto não entregue pela transportadora, a empresa poderá avisá-lo de um possível atraso, pois foi alertada pelo sistema de rastreo. Também terá insights analisando os relatórios gerados pelo sistema sobre o histórico de entregas.

1. Requisitos Funcionais (RF)

Gestão de Encomendas e Rastreamento

- **RF01 - Cadastro de Pedidos:** O sistema deve permitir o cadastro manual de dados de envio e códigos de rastreo das encomendas, onde o sistema irá identificar automaticamente de qual transportadora o código se refere.
- **RF02 - Consulta Automática de Status:** O sistema deve consultar de forma periódica e assíncrona o status das encomendas junto às APIs das transportadoras (ou receber atualizações via *webhooks*).
- **RF03 - Histórico Unificado:** O sistema deve padronizar e armazenar a linha do tempo (*timeline*) de todos os eventos de movimentação de cada pacote, independentemente da transportadora.

Alertas Proativos e Comunicação

- **RF04 - Detecção de Anomalias/Atrasos Preditivos:** O sistema deve identificar automaticamente inconsistências no fluxo de transporte (ex: pacotes parados mais tempo que o esperado na mesma filial, tentativas de entrega frustradas ou rotas desviadas) antes do prazo limite expirar.
- **RF05 - Configuração de Gatilhos de Notificação:** A Empresa Cliente deve conseguir parametrizar quais eventos e atrasos disparam alertas proativos automaticamente para o Cliente Final.
- **RF06 - Painel de Alertas Operacionais:** O sistema deve exibir uma central de incidentes.

Relatórios e Insights Analíticos

- **RF07 - Dashboard de Desempenho por Transportadora:** O sistema deve gerar relatórios comparativos com métricas de qualidade de entrega (ex: tempo médio de trânsito, taxa de pontualidade, índice de sinistros e reentrega) por transportadora e região.
- **RF09 - Exportação e Filtros Analíticos:** O sistema deve permitir filtrar relatórios por período, status, região e transportadora.

Gestão da Conta e Autenticação

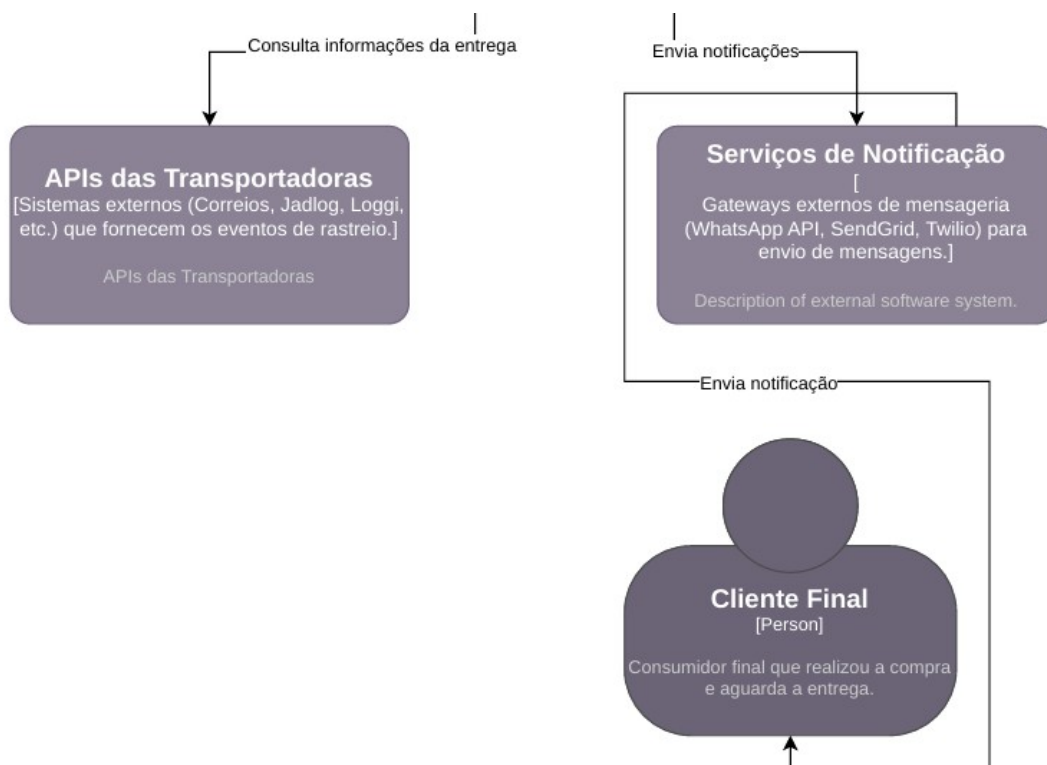
- **RF10 - Gestão Multi-tenant e Permissões:** O sistema deve isolar os dados de cada Empresa Cliente (SaaS *multi-tenant*) e permitir a gestão de usuários com diferentes níveis de acesso (Admin, Operador de Logística, Suporte).

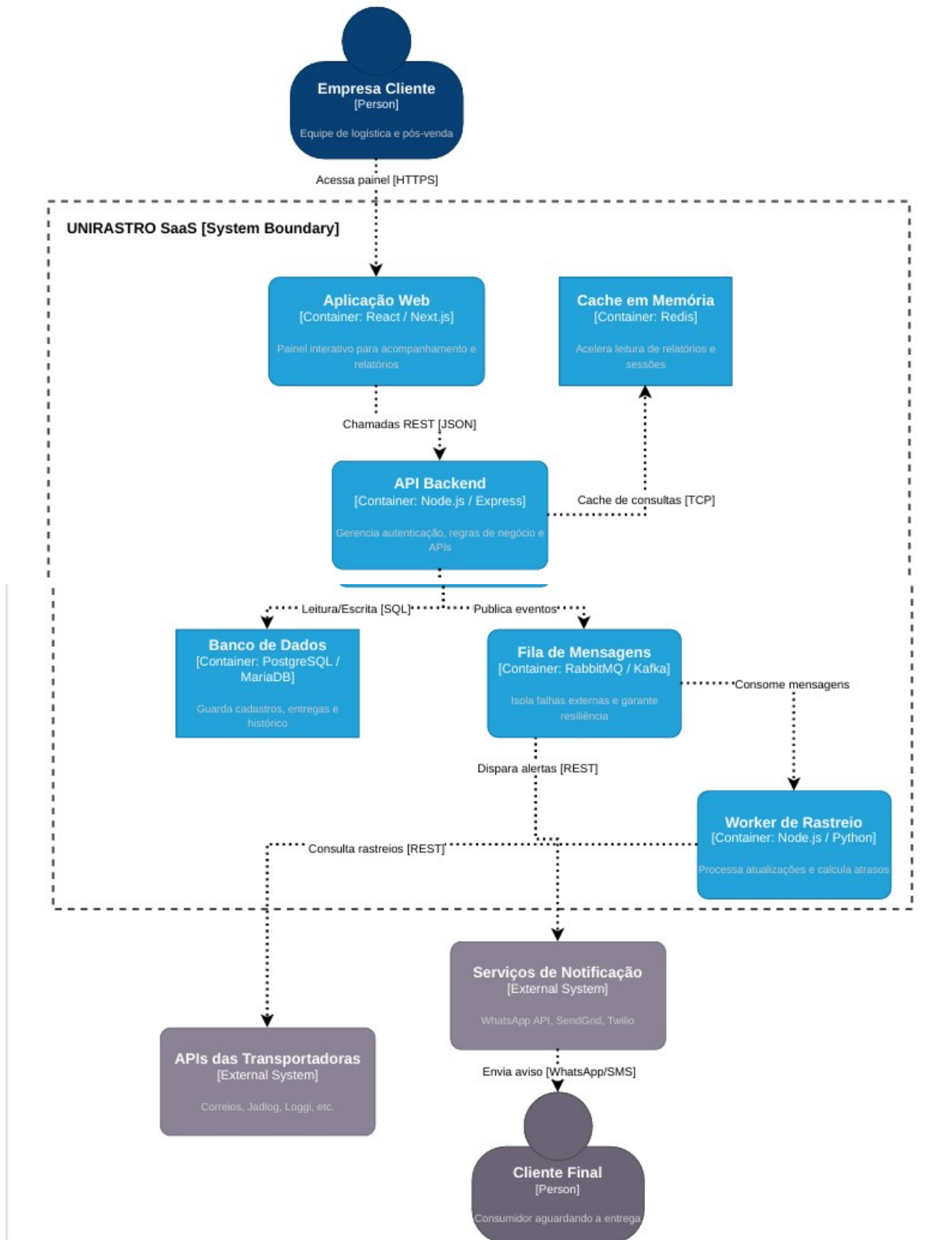
2. Requisitos Não Funcionais (RNF)

- **RNF01 - Arquitetura Cliente-Servidor:** O sistema deve ser estruturado sobre o padrão arquitetural Cliente-Servidor, no qual uma interface web/mobile (Cliente) interage de forma desacoplada com a API central do SaaS (Servidor) responsável pelo processamento de regras de negócio, persistência de dados e integrações.
- **RNF02 - Disponibilidade Alta com Redundância:** O SaaS deve manter um tempo de atividade (*uptime*) contínuo de **99,9%**, utilizando redundância de servidores e monitoramento ativo da infraestrutura, aceitando maior custo operacional e maior complexidade de implantação (pipelines de CI/CD automáticos).
- **RNF03 - Resiliência e Consistência Eventual:** O sistema deve garantir resiliência e tolerância a falhas na integração com transportadoras através do uso de filas de mensagens (ex: Kafka) e mecanismos de *retry* com *exponential backoff*, abrindo mão da consistência imediata dos dados em favor da alta disponibilidade.
- **RNF04 - Desempenho via Memória em Cache:** O sistema deve utilizar estratégias de *caching* em memória (ex: Redis) para otimizar o tempo de resposta do painel e acelerar a recuperação de dados frequentes, aceitando um consumo mais elevado de memória RAM na infraestrutura de nuvem.

Modelo C4 - Falta C3 e C4







Deve ser especificado também, nessa mesma documentação, qual o padrão de arquitetura será usado e o porquê. Caso decida-se por mais de um padrão, definia-o também.

Para atender aos requisitos de alta **Disponibilidade**, **Resiliência** e **Desempenho**, o **UNIRASTRO** adotará uma arquitetura híbrida **Cliente-Servidor Orientada a Eventos (Event-Driven)** com o padrão **API Gateway e Workers especializados**. O modelo **Cliente-Servidor** garante a separação total entre o painel de gestão (*Frontend*) e as regras de negócio (*Backend*), enquanto a **Orientação a Eventos com Filas de Mensagens (RabbitMQ/Kafka)** desacopla o processamento assíncrono das consultas e alertas; essa combinação é indispensável para suportar as constantes falhas e instabilidades das APIs externas das transportadoras sem comprometer o *uptime* da plataforma, assegurando que as notificações proativas de atraso e os relatórios analíticos sejam processados com rapidez, segurança e alta tolerância a falhas.